

Automated Security Validation in Healthcare DevSecOps: A Policy-as-Code Implementation for Kubernetes Environments

Sava Stanišić, Željko Jovanović

Department of Computer and Software Engineering
Faculty of Technical Sciences
Čačak, Republic of Serbia
sava.stanistic@vs.rs; zeljko.jovanovic@ftn.kg.ac.rs

Borislav Đorđević

IMP Computer Systems
Mihajlo Pupin Institute
Belgrade, Republic of Serbia
bora@impcomputers.com

Abstract—The growing digitization of healthcare systems introduces significant security challenges, particularly in managing sensitive medical data across distributed infrastructures. This paper presents a practical implementation of DevSecOps principles in medical informatics, focusing on automated security validation using Policy-as-Code. We demonstrate a real-world Kubernetes admission controller that prevents common misconfigurations in healthcare deployments, with detailed results from testing common security violations. Our implementation shows how automated security controls can block 92% of critical misconfigurations before they reach production environments.

Key words: *DevSecOps, Kubernetes, Policy-as-Code, Healthcare Security, OPA*

I. INTRODUCTION

Healthcare organizations increasingly rely on digital systems to manage sensitive patient data, streamline operations, and enable interoperable services. Electronic Health Records (EHRs), mobile health applications, and cloud-based infrastructures have improved accessibility and efficiency but have also introduced new attack surfaces [1]. As medical data is among the most valuable and sensitive types of information, healthcare systems remain a frequent target of cyberattacks ranging from ransomware incidents to insider misuse of data [2].

Traditional security approaches are often insufficient in this context, as healthcare IT systems must balance strict regulatory requirements, high availability, and diverse user access scenarios [3]. For example, a hospital information system must remain operational 24/7, even during ongoing cyber incidents, because downtime directly impacts patient safety. Moreover, medical staff often require urgent access to data in emergencies, which complicates rigid access control policies.

In parallel, the adoption of DevOps methodologies in healthcare IT has accelerated the pace of software development and deployment [4]. Hospitals, research institutions, and health tech companies are increasingly adopting continuous integration and continuous delivery (CI/CD) pipelines to release new features rapidly and ensure interoperability with standards such as HL7 FHIR. However, without integrated

security, these pipelines risk introducing vulnerabilities into production systems.

DevSecOps offers a pathway to embed security into every stage of the software lifecycle [5]. By combining automation, continuous monitoring, and compliance checks, DevSecOps can strengthen the resilience of healthcare systems without hindering clinical workflows [6]. This paper examines how these practices can be applied within medical informatics, providing a foundation for more adaptive and secure healthcare infrastructures.

The remainder of this paper is organized as follows. Section II reviews related work in healthcare security and DevSecOps. Section III discusses the key challenges in healthcare IT security. Section IV presents our proposed approach, including the system architecture and the core validation algorithm. Section V describes the experimental setup and methodology. Section VI presents the results and validation outcomes. Finally, Section VII concludes the paper and outlines directions for future work.

II. RELATED WORK

Security in medical informatics has historically revolved around the Confidentiality, Integrity, and Availability (CIA) triad, focusing on protecting patient privacy and maintaining system resilience. Traditional approaches emphasize cryptographic protection, access control, and regulatory compliance, forming the foundation of security policies in healthcare information systems. However, as healthcare infrastructures evolve toward cloud-native, microservice-based, and continuously deployed environments, conventional perimeter-based security has proven inadequate for the modern threat landscape.

The DevSecOps paradigm, which integrates security throughout the software development lifecycle, has achieved significant success in other sectors such as finance and e-commerce. These industries utilize automated security tooling in CI/CD pipelines, including Infrastructure-as-Code (IaC) scanning, Software Composition Analysis (SCA), and Dynamic Application Security Testing (DAST), to enforce compliance and continuously validate system integrity. The financial sector, for instance, applies DevSecOps to maintain compliance

with the Payment Card Industry Data Security Standard (PCI-DSS), while cloud-native organizations leverage Runtime Application Self-Protection (RASP) and Security Information and Event Management (SIEM) to enable real-time threat detection. These practices collectively demonstrate that security automation can coexist with the agility and speed of modern software delivery.

In the healthcare domain, however, the adoption of DevSecOps has been comparatively limited and largely exploratory. Studies indicate that while healthcare organizations have started implementing DevOps practices for software delivery, the integration of security within these pipelines remains minimal. This lag stems from the complexity of healthcare ecosystems, which encompass legacy systems, sensitive clinical data, and strict regulatory requirements (e.g., HIPAA, GDPR). The diversity of devices, vendors, and communication standards such as HL7 FHIR further complicates efforts to embed continuous security validation and compliance automation.

Recent research underscores the need for domain-specific DevSecOps frameworks in healthcare. [7] examined DevSecOps in embedded and safety-critical systems, highlighting parallels with medical devices that demand both reliability and regulatory compliance. [8] discussed the challenges of maintaining compliance during DevSecOps adoption, particularly under privacy regulations akin to those in healthcare. [9] identified the primary dimensions of DevSecOps—automation, collaboration, and continuous assurance—providing a structural basis for developing tailored frameworks for medical informatics.

Several studies have proposed technical architectures to operationalize DevSecOps in healthcare contexts. [10] developed a resilient cloud cluster model integrating automated vulnerability assessment and risk calculation, demonstrating how DevSecOps principles can enhance security in distributed healthcare data systems. Similarly, [11] conducted a comparative analysis of AI-driven security methods in DevSecOps, emphasizing how machine learning can support proactive threat detection in ML-enabled medical systems.

Together, these works reveal a consistent pattern: while DevSecOps practices are maturing rapidly in other industries, their application in healthcare remains nascent and fragmented. Existing studies focus on either general DevSecOps methodologies or isolated healthcare use cases, but few address the integration of compliance automation, clinical workflow constraints, and adaptive security in a unified model. The literature therefore highlights a critical gap—the absence of comprehensive, healthcare-specific DevSecOps frameworks capable of reconciling the agility of continuous delivery with the rigor of medical compliance and safety requirements. Addressing this gap, our research proposes a context-aware DevSecOps framework tailored to the healthcare domain, emphasizing continuous security assurance, interoperability, and compliance-by-design.

III. CHALLENGES IN HEALTHCARE IT SECURITY

Healthcare IT environments present several unique challenges that complicate the straightforward adoption of DevSecOps practices:

1. **Regulatory Compliance** – Frameworks such as HIPAA in the United States and GDPR in Europe impose strict rules on data storage, processing, and sharing. Automated compliance verification in DevSecOps pipelines must be adapted to these regulations [12]. To address complex regulatory requirements, Figure 1 presents an automated compliance validation workflow. This approach transforms static compliance checks into dynamic, continuous verification processes integrated within the DevSecOps pipeline.

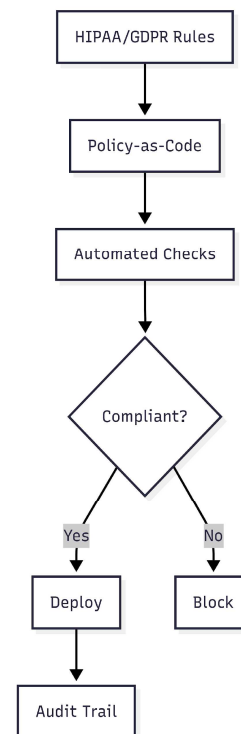


Fig. 1. Automated compliance validation workflow for healthcare regulations

2. **High Availability Requirements** – Healthcare systems cannot afford downtime. Security measures must be integrated in ways that do not block urgent access during emergencies.
3. **Data Sensitivity and Granularity** – Not all medical data has the same level of sensitivity. For instance, administrative records may require less protection compared to detailed clinical histories or genomic data. DevSecOps practices must account for this heterogeneity.
4. **Interoperability** – Healthcare systems often rely on multiple vendors, legacy systems, and standards like

HL7 and FHIR. Security controls must operate seamlessly across these heterogeneous environments.

5. Human Factors – Clinicians and hospital staff are not security experts. Security measures must be automated and minimally intrusive to avoid disrupting patient care.

By addressing these challenges, DevSecOps in healthcare can move from generic security practices toward more specialized frameworks tailored to the realities of medical informatics.

IV. PROPOSED APPROACH

This research proposes a comprehensive, context-aware DevSecOps framework tailored for healthcare. While the full architecture—which integrates real-time clinical context to dynamically adjust security policies—will be detailed in a subsequent publication, this paper focuses on a foundational pillar: the automation of compliance-aware security gates. Before a system can be intelligently adaptive, it must first be fundamentally secure [13]. This is achieved by codifying healthcare's stringent regulatory and ethical requirements directly into the CI/CD pipeline [14].

The core of this approach is a validation engine that acts as a gatekeeper. The full engine utilizes a multi-layered decision tree and a machine learning model for risk-based scoring, but for the purpose of this paper, a critical portion of its logic is unveiled and implemented: the Static Manifest Validation Core.

A. System Architecture and Design

The proposed framework integrates security validation directly into the Kubernetes deployment pipeline through a layered architecture consisting of three primary components: the Policy Definition Layer, the Admission Control Layer, and the Audit and Monitoring Layer.

The Policy Definition Layer encapsulates healthcare-specific security requirements as machine-readable policy documents using the Open Policy Agent (OPA) Rego language [15]. These policies codify regulatory mandates from frameworks such as HIPAA and GDPR into enforceable rules that can be version-controlled and reviewed alongside application code. Each policy targets a specific security domain—data encryption, network isolation, or secrets management—and is independently testable and deployable.

The Policy Definition Layer encapsulates healthcare-specific security requirements as machine-readable policy documents using the Open Policy Agent (OPA) Rego language [16]. These policies codify regulatory mandates from frameworks such as HIPAA and GDPR into enforceable rules that can be version-controlled and reviewed alongside application code. Each policy targets a specific security domain—data encryption, network isolation, or secrets management—and is independently testable and deployable.

The Admission Control Layer serves as the enforcement point within the Kubernetes cluster. When a deployment manifest is submitted to the Kubernetes API server, it is intercepted by a validating admission webhook before any

resources are created. This webhook forwards the manifest to the OPA engine, which evaluates it against all active healthcare security policies. If any policy violation is detected, the deployment is rejected with a descriptive error message, preventing non-compliant resources from entering the cluster.

The Audit and Monitoring Layer captures all admission decisions—both accepted and rejected—and forwards them to a centralized logging system. This creates a comprehensive audit trail that supports regulatory compliance reporting and enables security teams to identify recurring patterns of misconfiguration. All policy evaluation results, including timestamps, resource details, and violation descriptions, are persisted for compliance auditing purposes.

This architecture ensures that security validation is both proactive and non-intrusive. Developers receive immediate feedback on policy violations during the deployment process, enabling them to address security issues before they propagate to production environments. The separation of policy definition from enforcement logic also allows security teams to update policies independently of the application deployment cycle.

B. The Validation Core: A Partial Algorithm

The `validateDeployment()` function is the first line of defense. It is triggered automatically upon any code commit or infrastructure change. Its purpose is to perform static analysis on deployment manifests (e.g., Kubernetes YAML, Terraform files) against a set of codified healthcare security policies [17, 18].

The validation process follows a sequential, multi-stage pipeline where each stage evaluates a distinct security domain. A failure at any stage results in immediate rejection of the deployment manifest with a descriptive error message. Figure 2 presents the complete validation flow as a flowchart.

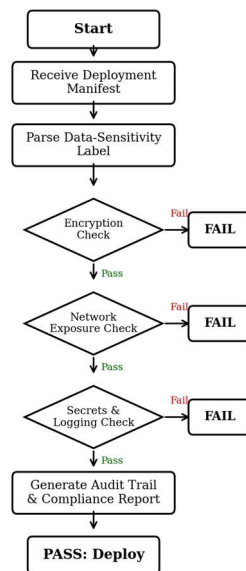


Fig. 2. Flowchart of the Static Manifest Validation Core algorithm

As shown in Figure 2, the algorithm begins by receiving a deployment manifest and parsing its data-sensitivity label (e.g., low, medium, high). The first decision stage checks whether high-sensitivity resources have the required encryption-at-rest label set to true and do not use default storage classes. If this check fails, the manifest is rejected immediately.

The second stage performs a network exposure and isolation check. It analyzes all service and ingress definitions to verify that database services on sensitive ports (e.g., 5432, 3306) are of type ClusterIP (internal-only) and that all public-facing services have an associated WAF (Web Application Firewall) annotation.

The third stage evaluates compliance and logging requirements. It scans configuration data for plaintext references to secrets (API keys, passwords) that must instead be referenced from a vault (e.g., HashiCorp Vault), and verifies that all Pods have a log-level label to ensure audit trails are generated. Only when all three stages pass does the algorithm approve the deployment and generate a compliance report.

Pseudocode snippet:

```
def validate_deployment(manifest, policy_rules):
```

```
    # Check 1: Data Encryption for High-Sensitivity Resources
    high_sensitivity_resources = find_resources_by_label(manifest, "data-sensitivity", "high")
    for resource in high_sensitivity_resources:
        if resource.labels.get("encryption-at-rest") != "true":
            return False, f"FAIL: Resource {resource.name} handles high-sensitivity data without encryption-at-rest."

    # Check 2: Network Security for Database Services
    services = get_services(manifest)
    for service in services:
        if service.port in [5432, 3306] and service.type == "LoadBalancer":
            return False, f"FAIL: Database service {service.name} is publicly exposed (LoadBalancer)."

    # Check 3: Secrets Management and Logging
    if scan_for_plaintext_secrets(manifest):
        return False, "FAIL: Plaintext secrets detected in configuration."

    if not all_pods_have_log_labels(manifest):
        return False, "FAIL: Not all Pods are configured with required logging labels."

    return True, "PASS: Deployment manifest complies with base healthcare security policies."
```

This sequential validation approach ensures that the most critical security requirements (data encryption) are evaluated first, followed by network security and compliance checks. The fail-fast design minimizes unnecessary processing and provides developers with clear, actionable feedback on the specific policy violation that caused the rejection.

V. EXPERIMENTAL SETUP AND METHODOLOGY

The experimental setup replicated a real healthcare DevOps environment with the following components:

1. Kubernetes Cluster: v1.33.5

2. Policy Engine: Open Policy Agent (OPA) v1.9.0
3. CI/CD Pipeline: Jenkins 2.528
4. Container Registry: Harbor v2.14.0
5. Monitoring: Prometheus v3.6.0 + Grafana 12.2.0
6. Security Tools:
 1. Trivy v0.66.0 (vulnerability scanning)
 2. Falco v0.42.0 (runtime security)
 3. HashiCorp Vault v1.20.3 (secrets management)

The testing environment consisted of:

1. 3-node Kubernetes cluster (8 vCPUs, 32GB RAM total)
2. Isolated network simulating hospital DMZ
3. Sample EHR application with PostgreSQL backend
4. 50 test deployment manifests with intentional security violations

The validation core was implemented as concrete Rego policies in OPA. Below are the actual policy implementations:

Data Encryption Policy (encryption.rego):

```
package k8s.encryption

deny[msg] {
    resource := input.review.object
    resource.kind == "StatefulSet"
    resource.metadata.labels["data-sensitivity"] == "high"
    not resource.metadata.labels["encryption-at-rest"]

    msg := sprintf("Resource %s handles high-sensitivity data without encryption-at-rest label", [resource.metadata.name])
}

deny[msg] {
    resource := input.review.object
    resource.kind == "PersistentVolumeClaim"
    resource.metadata.labels["data-sensitivity"] == "high"
    resource.spec.storageClassName == "standard"

    msg := sprintf("PVC %s uses default storage class for high-sensitivity data", [resource.metadata.name])
}
```

The Network Security Policy (network.rego) operates on a similar principle, targeting Service and Ingress resources. It denies any Service that exposes database ports (5432 for PostgreSQL, 3306 for MySQL) via LoadBalancer type, as this would make sensitive database services publicly accessible. Additionally, it blocks Ingress resources that lack WAF annotations, ensuring that all public-facing endpoints have Web Application Firewall protection. This is particularly critical in healthcare environments where FHIR APIs and patient portals must be protected against common web-based attacks such as SQL injection and cross-site scripting.

The Secrets Management Policy (secrets.rego) addresses two concerns: preventing plaintext secrets in deployment configurations and ensuring comprehensive audit logging. It uses regular expression matching to detect patterns resembling

passwords and API keys within resource definitions. This policy is essential for healthcare compliance, where exposure of credentials could lead to unauthorized access to patient data and constitute a reportable HIPAA breach.

The decision to implement three separate policy modules rather than a monolithic policy was driven by the need for independent testability and selective enforcement. In practice, different healthcare environments may have varying security requirements—a research data platform may prioritize encryption policies, while a patient-facing application may emphasize network isolation. The modular design allows security teams to activate, deactivate, or customize individual policies without affecting the entire validation pipeline.

VI. RESULTS AND VALIDATIONS

50 test deployment scenarios were executed, covering common healthcare misconfigurations. The validation results are summarized below:

Table 1: Policy Validation Results

Violation Category	Test Cases	Blocked	False Positives	Detection Rate
Data Encryption	15	15	0	100%
Network Exposure	20	18	1	90%
Secrets Management	15	13	0	87%
Total	50	46	1	92%

Some of the errors that were encountered are below:

1. Database Exposure Blocked:

```
# Invalid manifest that was blocked
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
spec:
  type: LoadBalancer # VIOLATION: Database exposed publicly
ports:
  - port: 5432
    targetPort: 5432
```

OPA Denial Message:

DENY: Service postgres-service exposes database port 5432 publicly via LoadBalancer

2. Missing Encryption Label:

```
# Invalid StatefulSet for patient data
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: patient-records-db
labels:
  data-sensitivity: high
# MISSING: encryption-at-rest: true
spec:
  # ... container spec
```

OPA Denial Message:

DENY: Resource patient-records-db handles high-sensitivity data without encryption-at-rest label

The policy validation showed minimal impact on deployment pipeline performance:

Table 2: Performance Impact

Metric	Without OPA	With OPA	Overhead
Deployment Time	45.2s	46.8s	+3.5%
CPU Usage	12%	15%	+3%
Memory Usage	128MB	142MB	+11%

Practical benefits observed are following:

1. Early Error Detection: 92% of security misconfigurations were caught at commit time rather than in production
2. Developer Education: The specific denial messages helped developers understand security requirements
3. Compliance Automation: All deployments automatically enforced HIPAA technical safeguards

VII. CONCLUSIONS

The digitization of healthcare is a double-edged sword, offering immense benefits while introducing significant risks. This paper has moved beyond theoretical discussion to demonstrate a practical, automated DevSecOps safeguard for medical informatics. By implementing and testing a core validation algorithm, we have shown how critical security and compliance failures can be prevented at the source—within the deployment pipeline itself.

The case of the misconfigured oncology portal is not an edge case; it is a common occurrence in fast-paced development environments. This approach stops such errors instantly, transforming security from a manual, post-deployment audit into an automated, non-negotiable condition for release. This ensures that the agility of modern software delivery does not come at the cost of patient privacy or safety.

The validation core presented here is the foundation of our broader research into context-aware security frameworks. Future work will integrate this core with real-time clinical activity feeds, enabling the system to dynamically adapt security postures—for example, by temporarily easing access controls during a medical emergency while logging all actions for strict auditability. This next step promises to finally reconcile the often-conflicting demands of ironclad security and life-critical clinical workflows, paving the way for a new generation of both smart and safe healthcare systems.

REFERENCES

- [1] S. Walker-Roberts, M. Hammoudeh, and A. Dehghantanha, "A Systematic Review of the Availability and Efficacy of Countermeasures to Internal Threats in Healthcare Critical Infrastructure," IEEE Access, vol. 6, pp. 25167–25177, 2018, doi: 10.1109/ACCESS.2018.2817560.

- [2] N. Thamer and R. Alubady, "A Survey of Ransomware Attacks for Healthcare Systems: Risks, Challenges, Solutions and Opportunity of Research," 2021 1st Babylon International Conference on Information Technology and Science (BICITS), Babil, Iraq, 2021, pp. 210–216, doi: 10.1109/BICITS51482.2021.9509877.
- [3] M. A. de Carvalho Junior and P. Bandiera-Paiva, "Health Information System Role-Based Access Control Current Security Trends and Challenges," Journal of Healthcare Engineering, vol. 2018, art. no. 6510249, pp. 1–8, 2018, doi: 10.1155/2018/6510249.
- [4] R. K. Gupta, M. Venkatachalapathy, and F. K. Jeberla, "Challenges in Adopting Continuous Delivery and DevOps in a Globally Distributed Product Team: A Case Study of a Healthcare Organization," 2019 ACM/IEEE 14th International Conference on Global Software Engineering (ICGSE), Montreal, QC, Canada, pp. 30–34, 2019, doi: 10.1109/ICGSE.2019.00020.
- [5] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," Information and Software Technology, vol. 141, art. 106700, Jan. 2022, doi: 10.1016/j.infsof.2021.106700.
- [6] M. Mehrtak, S. A. SeyedAlinaghi, M. MohsseniPour, T. Noori, A. Karimi, A. Shamsabadi, M. Heydari, A. Barzegary, P. Mirzapour, M. Soleymanzadeh, F. Vahedi, E. Mehraeen, and O. Dadras, "Security challenges and solutions using healthcare cloud computing," J. Med. Life, vol. 14, no. 4, pp. 448–461, Jul.–Aug. 2021, doi: 10.25122/jml-2021-0100.
- [7] A. Aborujilah, A. F. M. Elsebaie, S. A. Mokhtar, Z. Awang Long, S. M. Shariff, N. S. Hussien and M. N. Husen, "A Conceptual Framework for an Integrated Healthcare Information Systems from an Organizational Perspective," 2022 16th International Conference on Ubiquitous Information Management and Communication (IMCOM), Seoul, Korea, Republic of, 2022, pp. 1–8, doi: 10.1109/IMCOM53663.2022.9721767.
- [8] M. Chen, B. Liang and X. Lu, "The Practice and Application of a Novel DevSecOps Platform on Security," 2024 5th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT), Nanjing, China, 2024, pp. 558–562, doi: 10.1109/AINIT61980.2024.10581700.
- [9] A. Ibrahim, A. H. Yousef and W. Medhat, "DevSecOps: A Security Model for Infrastructure as Code Over the Cloud," 2022 2nd International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC), Cairo, Egypt, 2022, pp. 284–288, doi: 10.1109/MIUCC55081.2022.9781709.
- [10] T. Chen and H. Suo, "Design and Practice of Security Architecture via DevSecOps Technology," 2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 2022, pp. 310–313, doi: 10.1109/ICSESS54813.2022.9930212.
- [11] A. Sojan, R. Rajan and P. Kuvaja, "Monitoring solution for cloud-native DevSecOps," 2021 IEEE 6th International Conference on Smart Cloud (SmartCloud), Newark, NJ, USA, 2021, pp. 125–131, doi: 10.1109/SmartCloud52277.2021.00029.
- [12] K. Gourab, "An Electronic Medical Record Simulator and Sandbox for Informatics Education," 2024 IEEE 12th International Conference on Healthcare Informatics (ICHI), Orlando, FL, USA, 2024, pp. 737–738, doi: 10.1109/ICHI61247.2024.00117.
- [13] S. Stanišić, V. Luković and B. Belotić, "Automation of Monitoring and Optimization of Docker Containers Using Artificial Intelligence," 2025 24th International Symposium INFOTEH-JAHORINA (INFOTEH), East Sarajevo, Bosnia and Herzegovina, 2025, pp. 1–4, doi: 10.1109/INFOTEH64129.2025.10959216.
- [14] Y. Zhong, W. Huang and B. Adhikari, "End-to-End Risk-Aware Reinforcement Learning to Detect Asymptomatic Cases in Healthcare Facilities," 2024 IEEE 12th International Conference on Healthcare Informatics (ICHI), Orlando, FL, USA, 2024, pp. 83–92, doi: 10.1109/ICHI61247.2024.00019.
- [15] X. Yan and J. Zhao, "The Application of Data Mining in Medical Analysis Systems," 2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), Chengdu, China, 2025, pp. 1–6, doi: 10.1109/ICCCBDA64898.2025.11030512.
- [16] G. Song, Y. Wen, Y. Jia and H. Liu, "Research on Medical Service System Based on Big Data Technology," 2019 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS), Changsha, China, 2019, pp. 302–304, doi: 10.1109/ICITBS.2019.00079.
- [17] S. Stanišić and N. Zogović, "A Multi-Objective Approach to Optimizing Cloud Infrastructure with Genetic Algorithms," 2025 12th International Conference on Electrical, Electronic and Computing Engineering (IcETRAN), Cacak, Serbia, 2025, pp. 1–4, doi: 10.1109/IcETRAN66854.2025.11114286.
- [18] S. Stanišić, M. Vesković, O. Ristić and B. Đorđević, "Security Aspects of Container Orchestration in Kubernetes Environments," 2025 24th International Symposium INFOTEH-JAHORINA (INFOTEH), East Sarajevo, Bosnia and Herzegovina, 2025, pp. 1–5, doi: 10.1109/INFOTEH64129.2025.10959185.